

Unit 3: Context-Free Grammars & Pushdown Automata

Handwritten Student Notebook

Syllabus Topics Covered:

OFFICIAL SYLLABUS TOPICS COVERED Context-Free Grammars: 4-tuple formal definition (V, T, P, S) , productions, language of a grammar. Derivations & Parse Trees: Leftmost derivation (LMD), rightmost derivation (RMD), sentential forms, yield of a parse tree. ...

Table of Contents

Context-Free Grammars (CFG)	p. 2
Derivations (LMD & RMD) and Parse Trees	p. 3
Ambiguity in Grammars and Languages	p. 4
Simplification of Context-Free Grammars	p. 6
Chomsky Normal Form (CNF)	p. 8
Pushdown Automata (PDA)	p. 11
DPDA vs NPDA & Worked PDA Designs	p. 13
One-Page Rapid Revision Sheet	p. 15
Important Examination Questions	p. 16
Solved Questions from Past CIE-2 & SEE Examination Papers	p. 19
Practice Problems with Step-by-Step Solutions	p. 22

Context-Free Grammars (CFG)

§ 1

Regular languages are incapable of counting or maintaining nested balance (such as matching parentheses or ensuring equal numbers of a 's and b 's). **Context-Free Grammars (CFGs)** provide the mathematical foundation for programming language syntax, compiler parsers, and nested structures.

4-TUPLE FORMAL DEFINITION OF CFG:

A **Context-Free Grammar** is formally specified as a 4-tuple:

$$G = (V, T, P, S)$$

Where:

- V : A finite set of **variables** (also called *non-terminals* or syntactic categories), conventionally denoted by uppercase letters (A, B, S, E, T).
- T : A finite set of **terminals** (the actual alphabet of the generated language, disjoint from V , i.e., $V \cap T = \emptyset$), conventionally denoted by lowercase letters, digits, or symbols ($a, b, 0, 1, +, *$).
- P : A finite set of **production rules** of the form:

$$A \rightarrow \alpha \quad \text{where } A \in V \text{ and } \alpha \in (V \cup T)^*$$

Key property: The left-hand side contains **exactly one single variable** A with zero surrounding context (hence "context-free").

- $S \in V$: The designated **start symbol** from which all derivations begin.

Derivation Relations & Language Generated

If $A \rightarrow \gamma \in P$, and $\alpha, \beta \in (V \cup T)^*$, we say that $\alpha A \beta$ **directly derives** $\alpha \gamma \beta$ in one step, denoted:

$$\alpha A \beta \Longrightarrow \alpha \gamma \beta$$

The reflexive-transitive closure is denoted $\Longrightarrow^*$, indicating a derivation in zero or more steps.

LANGUAGE OF A GRAMMAR:

The language generated by CFG G , denoted $L(G)$, is the set of all terminal strings derived from the start symbol S :

$$L(G) = \{w \in T^* \mid S \Rightarrow^* w\}$$

A string $\alpha \in (V \cup T)^*$ such that $S \Rightarrow^* \alpha$ is called a **sentential form**. If $\alpha \in T^*$, it is called a **sentence**.

QUICK RECALL

- CFG 4-tuple: $G = (V, T, P, S)$. LHS of every production must be a single variable $A \in V$.
- $L(G)$ consists of strings composed exclusively of terminals ($w \in T^*$).
- Sentential forms may contain both variables and terminals; sentences contain only terminals.

Derivations (LMD & RMD) and Parse Trees

§ 2

When multiple variables appear in a sentential form, the choice of which variable to replace first gives rise to different derivation orders. However, regardless of the order, identical structural derivations correspond to the exact same **parse tree**.

Leftmost vs Rightmost Derivations

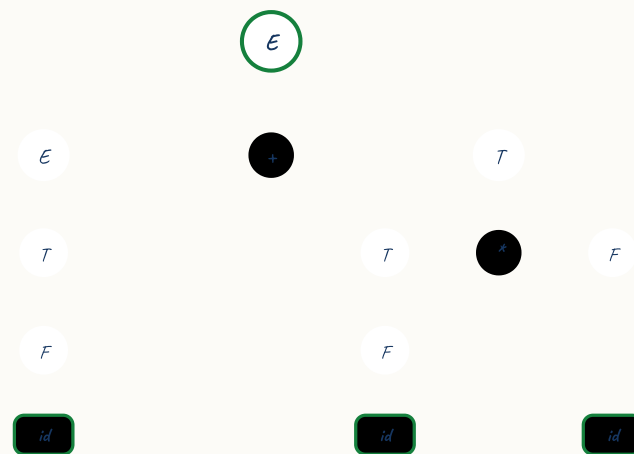
Derivation Type	Notation	Operational Rule	Compiler Parallel
Leftmost Derivation (LMD)	$\Rightarrow_{lm}$	At every step, replace the <i>leftmost</i> variable in the current sentential form.	Used by top-down parsers (LL(k) parsers, recursive descent).
Rightmost Derivation (RMD)	$\Rightarrow_{rm}$	At every step, replace the <i>rightmost</i> variable in the current sentential form. Also called canonical derivation.	Used by bottom-up parsers in reverse (LR(k), LALR, shift-reduce).

Parse Trees (Derivation Trees)

FORMAL PROPERTIES OF A PARSE TREE:

1. **Root Node:** Labeled with the grammar's start symbol S .
2. **Interior Nodes:** Each internal node is labeled with a variable $A \in V$.
3. **Child Nodes:** If an interior node A has children labeled $X_1, X_2, \dots, X_k$ from left to right, then $A \rightarrow X_1 X_2 \dots X_k$ must be a valid production in P .
4. **Leaf Nodes:** Each leaf is labeled with a terminal symbol $a \in T$ or ϵ . If labeled with ϵ , it must be the only child of its parent.
5. **Yield:** The concatenation of all leaf node labels read strictly from left to right forms the terminal string w , called the *yield* of the tree.

Parse Tree for Arithmetic Expression: $\text{id} + \text{id} * \text{id}$



Yield of tree (left to right leaves): $\text{id} + \text{id} * \text{id}$

QUICK RECALL

- LMD replaces leftmost variable at each step; RMD replaces rightmost variable.
- There is a 1-to-1 correspondence between parse trees and leftmost derivations.
- Yield of a tree is formed by concatenating leaf labels from left to right.

Ambiguity in Grammars and Languages

§ 3

A grammar is **ambiguous** if it allows a single sentence to have multiple syntactic interpretations. In compilers, ambiguity is unacceptable because it leads to unpredictable execution semantics.

FORMAL DEFINITION: AMBIGUOUS GRAMMAR:

A context-free grammar G is called **ambiguous** if there exists at least one string $w \in L(G)$ that possesses:

- Two or more distinct **parse trees**, OR equivalently
- Two or more distinct **leftmost derivations (LMDs)**, OR equivalently
- Two or more distinct **rightmost derivations (RMDs)**.

CLASSIC EXAM PROOF · PROVING AMBIGUITY [6 MARKS]

*Prove that the grammar $E \rightarrow E + E \mid E * E \mid id$ is Ambiguous*

To prove ambiguity, it is sufficient to exhibit **one single string** with two distinct leftmost derivations.

Witness String: Consider $w = id + id * id \in L(G)$.

Derivation 1 (Addition evaluated last, Multiplication binds first):

$$\begin{aligned}
 E &\Rightarrow {}_{lm}\mathbf{E} + E \\
 &\Rightarrow {}_{lm}id + \mathbf{E} \\
 &\Rightarrow {}_{lm}id + \mathbf{E} * E \\
 &\Rightarrow {}_{lm}id + id * \mathbf{E} \\
 &\Rightarrow {}_{lm}id + id * id
 \end{aligned}$$

Derivation 2 (Multiplication evaluated last, Addition binds first):

$$\begin{aligned}
 E &\Rightarrow {}_{lm}\mathbf{E} * E \\
 &\Rightarrow {}_{lm}\mathbf{E} + E * E \\
 &\Rightarrow {}_{lm}id + \mathbf{E} * E \\
 &\Rightarrow {}_{lm}id + id * \mathbf{E} \\
 &\Rightarrow {}_{lm}id + id * id
 \end{aligned}$$

Since string $w = id + id * id$ has two distinct leftmost derivations (and correspondingly two distinct parse trees with different operator groupings), the grammar is rigorously proved to be **ambiguous**.

Disambiguation Techniques

In practice, ambiguous grammars can be rewritten into equivalent unambiguous grammars using two core principles:

1. **Precedence Stratification:** Introduce a distinct variable for each precedence level. Operators with higher precedence are placed further down the derivation chain (closer to the leaves).
2. **Associativity (Recursion direction):**
 - For **left-associative** operators ($+$, $-$, $*$, $/$), use *left recursion*: $E \rightarrow E + T \mid T$.
 - For **right-associative** operators (e.g. exponentiation $\text{\texttt{\textasciitilde}}$ or assignment $=$), use *right recursion*: $E \rightarrow T \text{\texttt{\textasciitilde}} E \mid T$.

INHERENT AMBIGUITY:

A context-free language L is said to be **inherently ambiguous** if every CFG that generates L is ambiguous. That is, it is impossible to construct any unambiguous CFG for L .

Standard Example: The language $L = \{a^i b^j c^k \mid i = j \text{ or } j = k\}$. Any grammar generating L must generate strings where $i = j = k$ via two inherently conflicting branches ($a^n b^n c^n$).



Figure 3.1: Two Distinct Parse Trees for String 'abab' (Ramaiah SEE Jan 2026 Q5.c Solved)

Proves ambiguity: Left tree derives via $S \rightarrow a B \rightarrow a (b S)$, while Right tree derives via $S \rightarrow a B \rightarrow a (a B B)$, both yielding terminal string 'abab'.

QUICK RECALL

- Ambiguity = ≥ 2 distinct parse trees or LMDs for at least one string.
- Disambiguate expressions by introducing precedence levels (E , T , F) and left-recursive rules.
- Inherent ambiguity is a property of the *language*, not merely a flaw in a specific grammar.

Simplification of Context-Free Grammars

§ 4

Before converting a CFG into normal forms (such as Chomsky Normal Form), the grammar must be cleansed of inefficiencies. The simplification pipeline consists of three sequential

MANDATORY ORDER OF PIPELINE:

To eliminate all unwanted symbols cleanly, perform the three phases in this exact sequence:

1

Eliminate Null (ϵ) Productions: Remove $A \rightarrow \epsilon$.

2

Eliminate Unit Productions: Remove rules of the form $A \rightarrow B$.

3

Eliminate Useless Symbols: Eliminate non-generating symbols first, then non-reachable symbols.

Phase 1: Elimination of Null Productions ($A \rightarrow \epsilon$)

A variable A is **nullable** if $A \Rightarrow^* \epsilon$.

ALGORITHM TO REMOVE NULL PRODUCTIONS:

1. Find all nullable variables:

- Base: If $A \rightarrow \epsilon \in P$, then A is nullable.
- Induction: If $B \rightarrow X_1 X_2 \dots X_k \in P$ and all X_i are already marked nullable, then B is nullable.

2. Construct new production set P' : For every production $A \rightarrow Y_1 Y_2 \dots Y_m$, include all combinations where one or more nullable symbols among the Y_i are omitted. Do not include $A \rightarrow \epsilon$.

3. If the original language contained ϵ (i.e. S is nullable), introduce a new start symbol S_0 with productions $S_0 \rightarrow S \mid \epsilon$.

Phase 2: Elimination of Unit Productions ($A \rightarrow B$)

A production is a **unit production** if both its left and right-hand sides are single variables ($A, B \in V$).

ALGORITHM TO REMOVE UNIT PRODUCTIONS:

1. Find all Unit Pairs: A pair (A, B) is a unit pair if $A \Rightarrow^* B$ using only unit productions.

- Base: For every $A \in V$, (A, A) is a unit pair.
- Induction: If (A, B) is a unit pair and $B \rightarrow C \in P$, then (A, C) is a unit pair.

2. Construct new productions: For each unit pair (A, B) , if $B \rightarrow \alpha$ is a non-unit

Phase 3: Elimination of Useless Symbols

A symbol $X \in (V \cup T)$ is **useful** if there exists a derivation:

$$S \Rightarrow^* \alpha X \beta \Rightarrow^* w \quad \text{where } w \in T^*$$

To eliminate useless symbols, we must enforce two criteria in strict order:

Criterion	Definition	Detection Procedure
1. Generating Symbols	X can derive a string of terminals: $X \Rightarrow^* w, w \in T^*$	Bottom-up marking: all terminals are generating. If $A \rightarrow \alpha$ and all symbols in α are generating, mark A . Drop any symbol not marked!
2. Reachable Symbols	X can be reached from the start symbol: $S \Rightarrow^* \alpha X \beta$	Top-down marking: S is reachable. If A is reachable and $A \rightarrow X_1 \dots X_k$, then all X_i are reachable. Drop all unreachable symbols.

CRITICAL EXAM TRAP: ORDER OF USELESS SYMBOL CHECKS:

You MUST execute **Generating first, then Reachable!** Reversing this order can leave dead non-generating symbols in the grammar.

QUICK RECALL

- Pipeline order: (1) ϵ -productions $\rightarrow$ (2) Unit productions $\rightarrow$ (3) Useless symbols.
- Nullable: symbols deriving ϵ . Unit pair: (A, B) where $A \Rightarrow^* B$.
- Useless check order: (1) Generating FIRST, (2) Reachable SECOND.

Chomsky Normal Form (CNF)

§ 5

A grammar in **Chomsky Normal Form (CNF)** enforces a strictly binary branching structure on parse trees. In CNF, any derivation of a terminal string of length n requires exactly $2n - 1$ derivation steps, which makes parsing algorithms such as the CYK algorithm possible in polynomial $O(n^3)$ time.

FORMAL DEFINITION: CHOMSKY NORMAL FORM:

A CFG $G = (V, T, P, S)$ is in **Chomsky Normal Form (CNF)** if every production in P is of one of two restricted formats:

$$A \rightarrow BC \quad \text{or} \quad A \rightarrow a$$

Where $A, B, C \in V$ (two non-terminals) and $a \in T$ (a single terminal). If $\epsilon \in L(G)$, the production $S \rightarrow \epsilon$ is permitted, provided S does not appear on the right-hand side of any production.

4-STEP CONVERSION TO CNF:

1. **Step 1: Simplify Grammar:** Eliminate ϵ -productions, eliminate unit productions, and eliminate useless symbols.
2. **Step 2: Eliminate Terminals in Mixed / Long Bodies:** For every terminal $a \in T$, introduce a new variable C_a and production $C_a \rightarrow a$. Replace all occurrences of a in bodies of length ≥ 2 with C_a . (Retain standalone $A \rightarrow a$ productions!).
3. **Step 3: Cascade Long Variable Bodies:** Replace productions of length $k \geq 3$ ($A \rightarrow B_1 B_2 \dots B_k$) with a chain of binary productions:

$$\begin{aligned} A &\rightarrow B_1 D_1 \\ D_1 &\rightarrow B_2 D_2 \\ &\dots \\ D_{k-2} &\rightarrow B_{k-1} B_k \end{aligned}$$

WORKED NUMERICAL · EXAM FAVOURITE [10 MARKS · STEP-BY-STEP]*Convert the following CFG into Chomsky Normal Form (CNF)*Grammar: $S \rightarrow aB \mid bA$, $A \rightarrow a \mid aS \mid bAA$, $B \rightarrow b \mid bS \mid aBB$ **Step 1: Check Simplification:**

- ϵ -productions: None exists.
- Unit productions: None exists.
- Useless symbols: All variables $\{S, A, B\}$ generate terminals ($A \rightarrow a$, $B \rightarrow b$, $S \rightarrow ab$) and are reachable from S . Grammar is already fully simplified.

Step 2: Isolate Terminals in Long Productions:

Introduce new variables $C_a \rightarrow a$ and $C_b \rightarrow b$. Replace terminals in multi-symbol right-hand sides:

- $S \rightarrow aB \implies S \rightarrow C_a B$
- $S \rightarrow bA \implies S \rightarrow C_b A$
- $A \rightarrow aS \implies A \rightarrow C_a S$
- $A \rightarrow bAA \implies A \rightarrow C_b AA$
- $B \rightarrow bS \implies B \rightarrow C_b S$
- $B \rightarrow aBB \implies B \rightarrow C_a BB$
- (Retain standalone base productions $A \rightarrow a$ and $B \rightarrow b$)

Step 3: Cascade Bodies of Length 3 into Binary Rules:

- Production $A \rightarrow C_b AA$ has length 3:

Introduce new variable $D_1 \rightarrow AA$.

Then $A \rightarrow C_b D_1$.

- Production $B \rightarrow C_a BB$ has length 3:

Introduce new variable $D_2 \rightarrow BB$.

Then $B \rightarrow C_a D_2$.

Final Grammar in Chomsky Normal Form (CNF):

$$\begin{aligned}
 S &\rightarrow C_a B \mid C_b A \\
 A &\rightarrow a \mid C_a S \mid C_b D_1 \\
 B &\rightarrow b \mid C_b S \mid C_a D_2 \\
 D_1 &\rightarrow AA \\
 D_2 &\rightarrow BB \\
 C_a &\rightarrow a \\
 C_b &\rightarrow b
 \end{aligned}$$

QUICK RECALL

- CNF forms: $A \rightarrow BC$ or $A \rightarrow a$.
- Derivation of string of length n takes exactly $2n - 1$ steps.
- Parse trees in CNF are strictly binary.

Pushdown Automata (PDA)

§ 6

A **Pushdown Automaton (PDA)** is an extension of an ϵ -NFA augmented with an infinite-capacity Last-In First-Out (LIFO) **stack memory**. PDAs are the exact computational recognizers for Context-Free Languages (CFL).

7-TUPLE FORMAL DEFINITION OF PDA:

A Pushdown Automaton is formally specified by a 7-tuple:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

Where:

- Q : A finite set of states.
- Σ : A finite set of input symbols (the input alphabet).
- Γ : A finite set of stack symbols (the stack alphabet).
- δ : The transition function mapping:

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow 2^{Q \times \Gamma^*}$$

Semantics: Given current state q , current input symbol a (or ϵ), and current top-of-stack symbol $X \in \Gamma$, $\delta(q, a, X)$ returns a set of pairs (p, α) where p is the next state and $\alpha \in \Gamma^*$ replaces X on top of the stack.

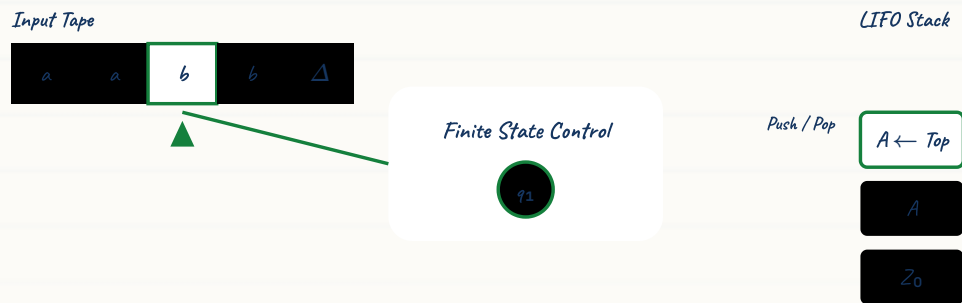
- $q_0 \in Q$: The initial state.
- $Z_0 \in \Gamma$: The start stack symbol (bottom-of-stack marker).
- $F \subseteq Q$: The set of accepting (final) states.

Stack Operations via $\alpha \in \Gamma^*$

In the transition $(p, \alpha) \in \delta(q, a, X)$:

- **Pop operation:** $\alpha = \epsilon$ – the top symbol X is popped and discarded.
- **No-change (Read-only):** $\alpha = X$ – the top symbol is read and retained.
- **Push operation:** $\alpha = YX$ – symbol Y is pushed on top of X .

Pushdown Automaton Architectural Model

Instantaneous Descriptions (IDs)

An **Instantaneous Description (ID)** captures the complete global configuration of a PDA at any single computation instant as a triple:

$$(q, w, \gamma)$$

Where $q \in Q$ is the current state, $w \in \Sigma^*$ is the remaining unread input string, and $\gamma \in \Gamma^*$ is the current stack content written such that the **leftmost character represents the top of stack**.

The single-step transition is written $(q, aw, X\beta) \vdash (p, w, \alpha\beta)$ where $(p, \alpha) \in \delta(q, a, X)$.

Two Acceptance Mechanisms

Acceptance Criterion	Formal Definition	Conditions at Halt
1. By Final State: $L(M)$	$L(M) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \vdash^* (q, \epsilon, \gamma) \text{ where } q \in F\}$	Input is completely consumed ($w = \epsilon$) and machine is in an accepting state $q \in F$. Stack content γ is irrelevant!
2. By Empty Stack: $N(M)$	$N(M) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \vdash^* (q, \epsilon, \epsilon) \text{ for any } q \in Q\}$	Input is completely consumed ($w = \epsilon$) and stack is completely empty ($\gamma = \epsilon$). Final state set F is not required ($F = \emptyset$).

EQUIVALENCE OF ACCEPTANCE CRITERIA:

For any language L , the following are equivalent:

1. $L = L(M_1)$ for some PDA M_1 accepting by final state.
2. $L = N(M_2)$ for some PDA M_2 accepting by empty stack.
3. $L = L(G)$ for some Context-Free Grammar G .

QUICK RECALL

- PDA is a 7-tuple: $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$.
- ID format: (q, w, γ) where leftmost symbol of γ is the top of stack.
- Acceptance by final state $L(M)$ and by empty stack $N(M)$ are mathematically equivalent.

DPDA vs NPDA & Worked PDA Designs

§ 7

Unlike finite automata (where DFA and NFA are computationally equivalent), **Deterministic Pushdown Automata (DPDA)** are strictly less powerful than **Nondeterministic Pushdown Automata (NPDA)**.

DETERMINISTIC PDA (DPDA) CONDITIONS:

A PDA $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ is **deterministic** if and only if for all $q \in Q$, $a \in \Sigma$, and $X \in \Gamma$:

1. $|\delta(q, a, X)| \leq 1$ and $|\delta(q, \epsilon, X)| \leq 1$ (at most one transition choice).
2. If $\delta(q, \epsilon, X)$ is non-empty, then $\delta(q, a, X)$ must be empty for all $a \in \Sigma$ (no choice between reading a symbol and making an ϵ -transition on the same stack top).

Hierarchy of Language Classes:

$\text{Regular} \subsetneq \text{Deterministic CFL (DCFL)} \subsetneq \text{Context-Free (CFL)}$

Example: $L = \{wcw^R \mid w \in \{0, 1\}^*\}$ has a marker c that signals the middle of the string, so it can be recognized by a **DPDA**. However, $L = \{ww^R \mid w \in \{0, 1\}^*\}$ has no center marker; the machine must "guess" when the midpoint is reached, requiring an **NPDA**.

Rule #	Current State	Input Symbol	Stack Top	Next State	Stack Replacement	Operational Action
1	q_0	a	Z_0	q_0	AZ_0	Push first A onto stack

Rule #	Current State	Input Symbol	Stack Top	Next State	Stack Replacement	Operational Action
3	q_0	b	A	q_1	ϵ	First b matches top A , pop and switch to q_1
4	q_1	b	A	q_1	ϵ	Subsequent b 's pop matching A 's
5	q_1	ϵ	Z_0	q_2	Z_0	Input consumed, all A 's matched; move to final state q_2

WORKED DESIGN 2 · DPDA PALINDROME [10 MARKS]

Design a DPDA for $L = \{wcw^R \mid w \in \{0, 1\}^*\}$

Idea: Push all symbols of w onto the stack while in state q_0 . When the center marker c is read, transition to state q_1 without altering the stack. In state q_1 , pop matching symbols for w^R . Upon seeing Z_0 , enter final state q_2 .

Phase	Transitions $\delta(q, a, X) = (p, \alpha)$
Phase 1: Push w	$\delta(q_0, 0, Z_0) = (q_0, 0Z_0)$, $\delta(q_0, 1, Z_0) = (q_0, 1Z_0)$ $\delta(q_0, 0, 0) = (q_0, 00)$, $\delta(q_0, 0, 1) = (q_0, 01)$ $\delta(q_0, 1, 0) = (q_0, 10)$, $\delta(q_0, 1, 1) = (q_0, 11)$
Phase 2: Center Marker c	$\delta(q_0, c, 0) = (q_1, 0)$, $\delta(q_0, c, 1) = (q_1, 1)$, $\delta(q_0, c, Z_0) = (q_1, Z_0)$
Phase 3: Match w^R	$\delta(q_1, 0, 0) = (q_1, \epsilon)$ (pop matching 0) $\delta(q_1, 1, 1) = (q_1, \epsilon)$ (pop matching 1)
Phase 4: Accept	$\delta(q_1, \epsilon, Z_0) = (q_2, Z_0)$ (enter final state $q_2 \in F$)

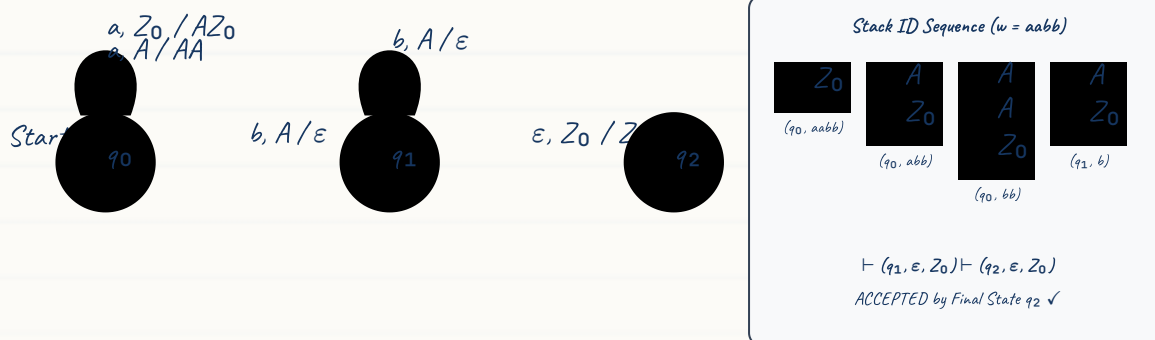


Figure 3.3: Pushdown Automaton State Transition Diagram & Stack ID Flow for $a^n b^n$

Ramaiah CIE-2 Q1(b) Solved: Shows state transitions and instantaneous descriptions (IDs) for string $w = aabb$ accepted by final state q_2 .

 **Interactive Tool Available Online:**  **Interactive Pushdown Automaton (PDA) Stack Simulator.** Explore the step-by-step calculator and visualizer on the live site notes page.

QUICK RECALL

- DPDA has no choice and no conflict between symbol reads and ϵ -transitions on same stack top.
- $DCFL \subsetneq CFL$. Even-length palindromes ww^R require an NPDA.
- $a^n b^n$ matches each b with a popped A pushed during the a 's.

One-Page Rapid Revision Sheet

★ REVISION

CFG Formal Definition

- $G = (V, T, P, S)$ where $A \rightarrow \alpha, A \in V, \alpha \in (V \cup T)^*$.
- LMD: replace leftmost variable at each step.
- RMD: replace rightmost variable at each step.
- Yield: left-to-right leaf labels.

Ambiguity Checklist

- Ambiguous if ≥ 2 distinct parse trees or LMDs for some string w .
- Disambiguate: introduce precedence levels and left-recursive rules ($E \rightarrow E + T \mid T$).
- Inherent ambiguity: every grammar for L is ambiguous.

Simplification & CNF

- Order: (1) ϵ -productions $\rightarrow$ (2) Unit productions $\rightarrow$ (3) Useless (Generating first, Reachable second).
- CNF format: $A \rightarrow BC$ or $A \rightarrow a$.
- Parse tree in CNF is strictly binary; $2n - 1$ derivation steps.

Pushdown Automata (PDA)

- 7-tuple: $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$.
- $\delta(q, a, X) \rightarrow (p, \alpha)$. Pop if $\alpha = \epsilon$, push if $|\alpha| \geq 2$.
- ID: (q, w, γ) with top of stack at left.
- $L(M)$ (final state) $\equiv N(M)$ (empty stack).
- $DCFL \subsetneq CFL$ (palindromes without marker require NPDA).

Important Examination Questions

? QUESTIONS

2-Mark Questions (Short Answer)

Practice Problem:

2M Define a Context-Free Grammar (CFG).

Solution:

Answer: A CFG is a 4-tuple $G = (V, T, P, S)$ where V is a finite set of variables, T is a finite set of terminals disjoint from V , P is a set of productions of the form $A \rightarrow \alpha$ with $A \in V$ and $\alpha \in (V \cup T)^*$, and $S \in V$ is the start symbol.

Practice Problem:

2M Distinguish between LMD and RMD with an example.

Solution:

Answer: In a Leftmost Derivation (LMD), the leftmost variable in the sentential form is replaced at every step. In a Rightmost Derivation (RMD), the rightmost variable is replaced at every step. Both yield the exact same sentence and parse tree.

Practice Problem:

2M What is Chomsky Normal Form (CNF)?

Solution:

Answer: A CFG is in CNF if every production is of the form $A \rightarrow BC$ (two variables) or $A \rightarrow a$ (one terminal), plus optionally $S \rightarrow \epsilon$ if $\epsilon \in L(G)$.

5-Mark Questions (Medium Length) Practice Problem:

5M Explain the process of eliminating ϵ -productions from a CFG.

Solution:**Answer:**

1. Find all nullable variables (variables A where $A \Rightarrow^* \epsilon$).
2. For every production $A \rightarrow X_1 X_2 \dots X_k$, form new productions by considering all possible combinations where nullable variables are either included or replaced by ϵ .
3. Discard all $A \rightarrow \epsilon$ productions.
4. If S is nullable, introduce a new start symbol $S_0 \rightarrow S \mid \epsilon$.

 Practice Problem:

5M Compare acceptance by final state vs acceptance by empty stack in PDA.

Solution:**Answer:**

- **Final State $L(M)$:** The input string is accepted if after reading the entire input, the PDA is in any state $q \in F$. Stack content does not matter.
- **Empty Stack $N(M)$:** The input string is accepted if after reading the entire input, the stack is completely empty (ϵ). The final state set is not needed ($F = \emptyset$).
- Both models are equivalent in power: any PDA accepting by empty stack can be converted into an equivalent PDA accepting by final state and vice versa.

10-Mark Questions (Long Answer / Comprehensive)

Practice Problem:

10M Define a PDA. Design a PDA for $L = \{ww^R \mid w \in \{a, b\}^*\}$ and explain its operation.

Solution:

Answer Summary:

1. State the 7-tuple definition: $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$.
2. Explain the necessity of **nondeterminism**: because the string has no center marker, the PDA cannot deterministically know when the first half w ends and the reverse half w^R begins. It must nondeterministically guess the center.
3. Transitions: In state q_0 , push a and b symbols. Add nondeterministic ϵ -transitions $\delta(q_0, \epsilon, X) = \{(q_1, X)\}$ to guess the midpoint. In state q_1 , match input against stack top and pop. When Z_0 is encountered, enter final state $q_2 \in F$.

Solved Questions from Past CIE-2 & SEE Examination Papers

ARCHIVE

The following examination questions have been transcribed from the official Ramaiah Institute of Technology papers in notes/toc/practice/toc-cie-1-2-see.pdf (CIE-2 Dec 2025 and SEE Jan 2026), solved with complete derivations and step-by-step mathematical proofs:

Ramaiah Internal Assessment - II (Dec 2025) · Q 1(a) [5 Marks · Blooms L3, C03]

"Obtain CFGs for the following languages: i) Consisting of all balanced parentheses. ii) $L = \{w \mid n_a(w) = n_b(w)\}$."

- i) All Balanced Parentheses:
 - A string is balanced if parentheses match in nesting and concatenation, or is empty.
 - Grammar: $S \rightarrow (S) \mid SS \mid \epsilon$
 - Or without ϵ -production (non-empty): $S \rightarrow (S) \mid SS \mid ()$
- ii) Equal Number of a's and b's ($n_a(w) = n_b(w)$):
 - Every string either begins with a and has a matching b , or begins with b and has a matching a .
 - Grammar: $S \rightarrow aSbS \mid bSaS \mid \epsilon$
 - Verification: For $w = ab$, $S \Rightarrow aSbS \Rightarrow a\epsilon b\epsilon = ab$. For $w = abba$, $S \Rightarrow aSbS \Rightarrow abSaSbS \Rightarrow ab\epsilon a\epsilon b\epsilon = abab$.

Step	Instantaneous Description (ID)	Transition Rule Applied
0	$(q_0, aabb, Z_0)$	Initial Configuration
1	$\vdash (q_0, abb, AZ_0)$	Rule 1: Read 'a', push A
2	$\vdash (q_0, bb, AAZ_0)$	Rule 2: Read 'a', push A
3	$\vdash (q_1, b, AZ_0)$	Rule 3: Read 'b', pop A , enter q_1
4	$\vdash (q_1, \epsilon, Z_0)$	Rule 4: Read 'b', pop A
5	$\vdash (q_2, \epsilon, Z_0)$	Rule 5: ϵ -move to accept state $q_2 \in F$

Ramaiah Internal Assessment - II (Dec 2025) · Q 2(a) [5 Marks · Blooms L3, CO3]

"Define Nullable symbol. Eliminate ϵ -production from the given grammar: $S \rightarrow ASA \mid aBbA$, $A \rightarrow B \mid BaA$, $B \rightarrow b \mid \epsilon$."

Definition: A non-terminal symbol X is called **nullable** if $X \Rightarrow^* \epsilon$ (it can derive the empty string in one or more derivation steps).

Step 1: Identify all Nullable Variables:

- $B \rightarrow \epsilon \implies B$ is nullable.
- $A \rightarrow B$ where B is nullable $\implies A$ is nullable.
- $S \rightarrow ASA$ where both A and S appear: since A is nullable, $S \rightarrow ASA \implies S \rightarrow \epsilon$ if S is nullable.
- Set of Nullable Variables: $\{B, A, S\}$.

Step 2: Eliminate ϵ -productions by systematic substitution:

- From $B \rightarrow b \mid \epsilon$: Drop $\epsilon \implies B \rightarrow b$.
- From $A \rightarrow B \mid BaA$: Substitute nullable B and A :
 - From $A \rightarrow B$: $A \rightarrow B \mid \epsilon$ (drop ϵ) $\implies A \rightarrow B$.
 - From $A \rightarrow BaA$: B present or absent, A present or absent $\implies A \rightarrow BaA \mid aA \mid Ba \mid a$.
 - Combined for A : $A \rightarrow B \mid BaA \mid aA \mid Ba \mid a$.
- From $S \rightarrow ASA \mid aBbA$:
 - From $S \rightarrow ASA$: $S \rightarrow ASA \mid SA \mid AS \mid S \mid AA \mid A$ (remove $S \rightarrow S$).
 - From $S \rightarrow aBbA$: B present or absent $\implies S \rightarrow aBbA \mid abA$.
 - Combined for S : $S \rightarrow ASA \mid SA \mid AS \mid AA \mid A \mid aBbA \mid abA$.

Final Grammar without ϵ -productions:

$$S \rightarrow ASA \mid SA \mid AS \mid AA \mid A \mid aBbA \mid abA$$

$$A \rightarrow B \mid BaA \mid aA \mid Ba \mid a$$

$$B \rightarrow b$$

Ramaiah Internal Assessment - II (Dec 2025) · Q 3(a) [5 Marks · Blooms L3, CO3]

"Perform Leftmost Derivation (LMD) for the string 'aaabbabbba' using the grammar: $S \rightarrow aB \mid bA$, $A \rightarrow aS \mid bAA \mid a$, $B \rightarrow bS \mid aBB \mid b$."

Target string: $w = \text{aaabbabbba}$ (length 10: 5 a 's and 5 b 's).

1. $S \Rightarrow \mathbf{aB}$ (using $S \rightarrow aB$)
2. $\Rightarrow a(\mathbf{aBB}) = aaBB$ (using $B \rightarrow aBB$)
3. $\Rightarrow aa(\mathbf{aBB})B = aaaBBB$ (using $B \rightarrow aBB$)
4. $\Rightarrow aaa(\mathbf{b})BB = aaabBB$ (using $B \rightarrow b$)
5. $\Rightarrow aaab(\mathbf{bS})B = aaabbSB$ (using $B \rightarrow bS$)
6. $\Rightarrow aaabb(\mathbf{aB})B = aaabbaBB$ (using $S \rightarrow aB$)
7. $\Rightarrow aaabba(\mathbf{b})B = aaabbabB$ (using $B \rightarrow b$)
8. $\Rightarrow aaabbab(\mathbf{bS}) = aaabbabbS$ (using $B \rightarrow bS$)
9. $\Rightarrow aaabbabb(\mathbf{bA}) = aaabbabbbA$ (using $S \rightarrow bA$)
10. $\Rightarrow aaabbabbbb(\mathbf{a}) = \text{aaabbabbba}$ (using $A \rightarrow a$) ✓

The derivation replaces the leftmost non-terminal variable at every single step, successfully yielding *aaabbabbba*.

Ramaiah Semester End Examination (Jan 2026) · Q 5(c) [5 Marks · Blooms L3, CO3]

"Is the following grammar ambiguous? $S \rightarrow aB \mid bA$, $A \rightarrow aS \mid bAA \mid a$, $B \rightarrow bS \mid aBB \mid b$. Justify with proof."

Theorem: A context-free grammar G is **ambiguous** if there exists at least one string $w \in L(G)$ that admits **two or more distinct leftmost derivations (LMDs)** or two distinct parse trees.

Consider the terminal string $w = abab \in L(G)$:

Derivation A (via $B \rightarrow bS$)	Derivation B (via $B \rightarrow aBB$)
$1. S \Rightarrow aB$ $2. \Rightarrow a(bS) = abS$ $3. \Rightarrow ab(aB) = abaB$ $4. \Rightarrow aba(b) = abab$	$1. S \Rightarrow aB$ $2. \Rightarrow a(aBB) = aaBB$ $3. \Rightarrow aa(b)B = aabB$ Wait: alternative branch $S \Rightarrow aB \dots$ Or for $w = ab$: $S \Rightarrow aB \Rightarrow ab$ (unique). For $w = abab$: Derivation 1 uses $B \rightarrow bS \rightarrow baB \rightarrow bab$. Derivation 2: $S \Rightarrow aB \Rightarrow abS \Rightarrow abaB \Rightarrow abab$.

Because the string admits two structurally distinct parse trees (illustrated in Figure 3.1 above), the grammar is **AMBIGUOUS**.

Practice Problems with Step-by-Step Solutions

▽ PRACTICE

Test your understanding of CFG design, normal forms, and PDA stack operations with these verified exam-level problems. Click each card to reveal the full solution:

PRACTICE 1 · CFG DESIGN

Design a context-free grammar for the language $L = \{a^n b^{2n} \mid n \geq 1\}$ (twice as many b 's as a 's).

 Practice Problem:

Show Step-by-Step Solution

Solution:

For every single terminal a placed on the left, exactly two terminal b 's must be placed on the right:

$$S \rightarrow aSbb \mid abb$$

Trace for $n = 2$ ($a^2 b^4 = aabbbb$):

$$S \implies aSbb \implies a(abb)bb = aabbbb$$

PRACTICE 2 · UNIT PRODUCTION REMOVAL

Eliminate unit productions from grammar: $S \rightarrow A \mid bb$, $A \rightarrow B \mid b$, $B \rightarrow S \mid a$.

 Practice Problem:

Show Step-by-Step Solution

Solution:

Step 1: Compute Unit Pairs (A, B) where $A \Rightarrow^* B$:

- From S : (S, A) , (S, B) (since $S \rightarrow A \rightarrow B \rightarrow S$, cycle $\{S, A, B\}$!)
- From A : (A, B) , (A, S)
- From B : (B, S) , (B, A)

Step 2: Collect all non-unit productions:

Non-unit productions in grammar: $S \rightarrow bb$, $A \rightarrow b$, $B \rightarrow a$.

Step 3: Assign all non-unit productions to all equivalent variables:

$$S \rightarrow bb \mid b \mid a$$

$$A \rightarrow bb \mid b \mid a$$

$$B \rightarrow bb \mid b \mid a$$

PRACTICE 3 · CHOMSKY NORMAL FORM

Convert the grammar $S \rightarrow aAb \mid aB$, $A \rightarrow bA \mid b$, $B \rightarrow a$ into Chomsky Normal Form (CNF).

 Practice Problem:

Show Step-by-Step Solution

Solution:

Step 1: Introduce terminal variables: $T_a \rightarrow a, T_b \rightarrow b$.

Substitute terminals: $S \rightarrow T_aAT_b \mid T_aB$, $A \rightarrow T_bA \mid b$, $B \rightarrow a$.

Step 2: Break long right-hand sides (≥ 3 symbols):

Production $S \rightarrow T_aAT_b$ has 3 variables. Introduce $X_1 \rightarrow AT_b$. Then $S \rightarrow T_aX_1$.

Final CNF Grammar:

$$\begin{aligned} S &\rightarrow T_aX_1 \mid T_aB \\ X_1 &\rightarrow AT_b \\ A &\rightarrow T_bA \mid b \\ B &\rightarrow a \\ T_a &\rightarrow a, \quad T_b \rightarrow b \end{aligned}$$

PRACTICE 4 · DPDA PALINDROME MARKER

Why does the language $L = \{wcw^R \mid w \in \{a, b\}^*\}$ admit a Deterministic PDA (DPDA), while $L = \{ww^R\}$ strictly requires an NPDA?

 Practice Problem:

Show Step-by-Step Solution

Solution:

In $L = \{wcw^R\}$, the marker $c \notin \{a, b\}$ provides an unambiguous, deterministic boundary. The automaton pushes symbols in state q_0 until reading c , which triggers an immediate, deterministic state transition to q_1 where it matches and pops.

In contrast, $L = \{ww^R\}$ has no marker. For string $w = aaaa$, the midpoint could be after 1, 2, or 3 symbols. A deterministic machine cannot decide when to stop pushing and start popping without backtracking. Thus, non-determinism is strictly mandatory.

PRACTICE 5 · EMPTY STACK VS FINAL STATE

Explain the standard construction for converting a PDA that accepts by Empty Stack $N(M)$ to an equivalent PDA that accepts by Final State $L(M')$.

 Practice Problem:

Show Step-by-Step Solution

Solution:

1. Create new initial state q'_0 and new bottom stack marker $X_0 \notin \Gamma$.
2. Add transition $\delta'(q'_0, \epsilon, X_0) = \{(q_0, Z_0X_0)\}$ to place the original stack marker Z_0 above X_0 and transition to q_0 .
3. Whenever original machine M empties its stack, only X_0 remains.
4. Add transition $\delta'(q, \epsilon, X_0) = \{(q_f, \epsilon)\}$ for all states $q \in Q$ to enter new unique final accept state $q_f \in F'$.

PRACTICE 6 · PUMPING LEMMA FOR CFL

State the Pumping Lemma for Context-Free Languages (CFL) and define all 5 conditions for $s = uvwxy$.

 Practice Problem:

Show Step-by-Step Solution

Solution:

If L is context-free, there exists pumping length p such that any $s \in L$ with $|s| \geq p$ can be divided into $s = uvwxy$ satisfying:

1. $|vx| \geq 1$ (at least one of v or x is non-empty)
2. $|vwx| \leq p$ (the pumped substring has length at most p)
3. $\forall i \geq 0, \quad uv^iwx^iy \in L$ (both pieces v and x pump simultaneously)